Improving Symbolic Regression with Interval Inversion and Constant Stabbing

Maarten Keijzer

July 2026

Abstract

1 Introduction

2 Semantic inversion using intervals

Semantic crossovers [**?**] uses analytical inversion to transform root level targets into surrogate targets that are used in a subsequent search for a good sub-tree for that position. While this preserves the location of the target value, it does not preserve the error structure in general, i.e., a subtree that is a better fit to the surrogate target according to some metric need not lead to a better overall error when inserted at the root level using that same metric. Only when the new sub-tree dominates the original sub-tree on all surrogate targets can a guaranteed improvement be realized.

Here, instead of using a squared error or related measure for judging different trees, interval bands are used. The target values are bound with one or more interval bands, each band representing a tolerance for being off-target.

Given a set of targets $[t_1, \ldots, t_n]$, the symbolic regression problem can be reformulated to hitting targets $[(t_1 - \epsilon, t_1 + \epsilon, \ldots, (t_n - \epsilon, t_n + \epsilon)]$, or in general hitting a set of intervals $[(l_1, u_1), \ldots, (u_n, l_n)]$

When we use a single band and global ϵ, This metric reduces to Koza's original hits metric [**?**].

When using multiple equally spaced bands $(\epsilon, 2\epsilon, \ldots)$, the fitness metric is roughly equivalent to ϵ-lexicase selection [**?**].

The advantage of using interval inversion is that hits are maintained. If a subtree hits (stabs) a surrogate target's interval, it is guaranteed that the overall tree will also hit the original target, and therefore scores a hit when inserted. The guarantee that the metric is invariant under inversion is absent with different loss functions.

However, using these surrogate targets for selecting sub-trees is not immediately useful. The efficiency gains when doing sub-tree selection at depth d in

the tree will only save dn evaluations compared to a naive insertion of these same sub-trees in the overall tree directly, and evaluate it at the root level.

3 Maximum overlapping intervals

Stabbing problems are studied in computational geometry to find the number of areas a ray, line, or general shape intersects with [**?**].

In symbolic regression with interval (surrogate) targets, the areas of interest are the (surrogate) target intervals, and trees try to hit (stab) as many intervals as possible. When all intervals are stabbed, a 100% hit rate is achieved.

Finding an interval that intersects with the maximum number of intervals from a set is an $O(nlogn)$ operation in the number of intervals considered; i.e.; given a set of interval targets, the algorithm finds a range of constants, each constant from that range stabbing the maximum number of intervals that are achievable.

Therefore, given a subtree T producing evaluations x against a surrogate target $[(l_1, u_1), ..., (l_n, u_n)]$, the maximum overlapping interval algorithm gives us a range of values $c \in (l, u)$ for which $x+c$ stabs most intervals, i.e., maximizes hits.

Given the invariance property of interval inversion on the number of hits, this makes it possible to find optimal additive constants anywhere in the tree. So instead of using unguided or gradient search to iteratively find a good constant, we can directly compute the optimal constant(s) in $O(nlogn)$ time. When this routine is used for crossover or mutation, it will ensure that the tree will be brought into the right range, stabbing at least one interval, and fully destructive crossovers are no longer possible.

Finding also the scale instead of just the intercept would be great, butl inear stabbing algorithms have complexity $O(n^2)$, and are therefore too expensive to run in the inner loop of a symbolic regression algorithm.

3.1 Selecting stabbing constants

The maximum interval overlap routine returns one or more intervals. Any value within that range is a valid constant for maximizing the number of hits. When selecting a constant, it is first checked if the value 0 is part of the range. If so, that value is chosen as it has a positive effect on structural complexity (x + 0 = x).

If that is not the case, another check is performed to see if there are integers that are optimal. If there are such integers, an integer is drawn uniformly from the intervals. This implements a preferential bias towards integer constants.

Finally, if there is no zero, nor integers in the range, a constant is drawn uniformly from the ranges.

4 Stabbing crossover and mutations

Using the machinery of interval inversion and constant stabbing, corresponding crossover and mutation operators are defined using an **aligned-insert** routine. It inserts the tree at a position and computes the optimal stabbing range from which a constant is drawn from the insertion location upwards towards the root. By drawing from the ranges, new optimal constants are tried out which might make the search in other branches easier (or harder.) Systematic varying the constants in the allowable range implements a form of coordinate descent on all constants in the tree.

A node in this system then has its normal payload together with an additive constant optimized by stabbing.

4.1 The aligned-insert algorithm

Every node carries an additive constant c, so a node with function f evaluates as $f(\cdot) + c$. The algorithm recurses from the root of R toward position i, propagating surrogate targets downward and updated evaluations upward, re-optimising each ancestor's additive constant on the way back.

MaxOverlap runs the $O(n \log n)$ interval-stabbing sweep and returns the optimal constant c^* (selected per Section 3.1). The total cost is $O(d \cdot n \log n)$ where d is the insertion depth, dominated by d calls to MaxOverlap. Nodes off the path are never re-evaluated.

5 Methods and materials

The previous sections laid out the main contribution of the paper. This section provides further algorithmic details and describes the experimental setup.

Structural complexity The structural complexity of a tree is computed as the number of nodes in a tree. As every node has an accompanying constant, an extra count for a non-zero constant is added to the structural complexity of a tree. Nodes with constants value zero thus get no penalty, while non-zero constants do.

Complexity stratification and Lexicase selection In our experiments, simple Lexicase selection [**?**] is used to select parents for crossover and mutation. Size controls are implemented by stratifying the population in size buckets, and lexicase selection is done inside the bucket. This not only reduces the complexity of lexicase selection by reducing the number of candidates, but makes the comparisons size-fair, i.e., parents are only competing against parents of the same size.

Size strata are chosen at random: the first stratum uniformly, and in the case of crossover, the second parent uniformly within plus or minus 10 complexity points from the first parent.

Algorithm 1 AlignedInsert$(N, D, i, \mathbf{x}, \mathcal{T})$

Require: recipient node N, donation subtree D, index i, inputs $\mathbf{x}$, interval targets $\mathcal{T} = [(l_j, u_j)]_{j=1}^{n}$
Ensure: updated node N', evaluations $\mathbf{v} \in \mathbb{R}^n$

if $i = 1$ **then** ▷ insertion point reached
  $\mathbf{v} \leftarrow \text{eval}(D, \mathbf{x}) - D.c$
  $c^*, \mathbf{v} \leftarrow \text{MaxOverlap}(\mathcal{T} - \mathbf{v})$
  return $D.\text{withConstant}(c^*),\ \mathbf{v} + c^*$
end if
if N is a **UnaryNode** with function g **then**
  $\mathcal{T}' \leftarrow g^{-1}(\mathcal{T} - N.c)$ ▷ interval inverse, pointwise
  $N'_{\text{child}}, \mathbf{v}_{\text{child}} \leftarrow \text{AlignedInsert}(N.\text{child}, D, i-1, \mathbf{x}, \mathcal{T}')$
  $\mathbf{v} \leftarrow g(\mathbf{v}_{\text{child}})$
else if N is a **BinaryNode** with function $\oplus$ and $i \le 1 + |N.\text{left}|$ **then**
  $\mathbf{v}_R \leftarrow \text{eval}(N.\text{right}, \mathbf{x})$
  $\mathcal{T}' \leftarrow \text{leftInverse}(\oplus,\ \mathcal{T} - N.c,\ \mathbf{v}_R)$
  $N'_{\text{left}}, \mathbf{v}_L \leftarrow \text{AlignedInsert}(N.\text{left}, D, i-1, \mathbf{x}, \mathcal{T}')$
  $\mathbf{v} \leftarrow \mathbf{v}_L \oplus \mathbf{v}_R$
else ▷ insertion is in right subtree
  $\mathbf{v}_L \leftarrow \text{eval}(N.\text{left}, \mathbf{x})$
  $\mathcal{T}' \leftarrow \text{rightInverse}(\oplus,\ \mathcal{T} - N.c,\ \mathbf{v}_L)$
  $N'_{\text{right}}, \mathbf{v}_R \leftarrow \text{AlignedInsert}(N.\text{right}, D, i-1-|N.\text{left}|, \mathbf{x}, \mathcal{T}')$
  $\mathbf{v} \leftarrow \mathbf{v}_L \oplus \mathbf{v}_R$
end if
$c^*, \mathbf{v} \leftarrow \text{MaxOverlap}(\mathcal{T} - \mathbf{v})$ ▷ re-optimise this node's constant
return $N'.\text{withConstant}(c^*),\ \mathbf{v} + c^*$

The resulting offspring is placed in the stratum corresponding to their structural complexity. After this, the stratum with most individuals is selected. An inverse lexicase is held in the stratum to identify a member for deletion. This keeps the strata well-populated with an equal number of individuals in each stratum.

Initialization To create an initial population, probabilistic tree creation [**?**] is used. If population size allows, every straturm is initialized with the same number of initial trees. There is no attempt to find optimal constants of these trees. THey are initialized as regular trees with random constants drawn from a Cauchy distribution (randn()/randn()).

5.1 Estimating computational effort

Comparing different symbolic regression systems for their efficiency to reach a certain level of performance is not straightforward. Standard metrics such as the number of individuals processed ignore important factors such as the number of fitness cases, the number of evaluations per individual, and the complexity of the individuals. A run that keeps average population size low can run many orders faster than a run that is allowed to bloat, yet in individuals processed they can be considered equal.

Therefore, many comparisons are done on wall-clock time [**?**], but this is hardware dependent, language dependent, implementation dependent, and can be affected by many other factors. At best, this gives a rough indication of efficiency, but it is not a reliable measure. Although wall-clock speed is important

A natural measure for computational effort is the number of evaluations performed. It would therefore be tempting to use the size of the population times the number of fitness cases as a measure of effort. Every new individual processed would be considered to be freshly evaluated. However, evaluations can be cached, and dramatic speedups can be achieved doing this, particularly when caching is used with algorithms that heavily share subtrees (e.g., use subtree crossover almost exclusively) [**?**].

A useful observation is that under assumptions of optimal caching, only new subtrees need to be evaluated. It is furthermore useful to ignore duplication effects where the same subtree is inserted at the same location in the same tree, and assume that every subtree inserted in a tree will lead to new evaluations from the insertion point upwards to the root. This was denoted by d above.

Therefore, a subtree crossover at depth d will add dn to the overall evaluation count, and a subtree mutation inserting an (assumed new and unique) subtree of size s at depth d will add $(d + s)n$ to the overall evaluation count.

Tracking all these insertion points and subtree sizes might be intrusive for an implementation, therefore a further, theoretically justified, simplification is used. Under assumptions of uniform selection of insertion points, together with a mutation operator that on average replaces a subtree with the same size as the original, the average depth of insertion as well as the average size of the inserted subtree can be tracked by keeping track of total path length and size

of individual trees. As shown in [**?**], the average depth of a tree is equal to the average size of the tree, and are equal to the total path length divided by the total size of the tree.

Using this, the expected depth, $\bar{d}$ for an event can be directly computed from total path length and size of the trees undergoing crossover and mutation.

In the algorithm studied here, each aligned insert requires $\bar{d}n \log n$ evaluations, which is a factor of $\log n$ more expensive than standard insertion. That is the computational effort that will be tracked in the experiments, and the question is whether the improved search efficiency of the aligned insert will outweigh the increased cost per insertion. Wall-clock time is also tracked as a control.

So our bookkeeping is the following. For regular crossover, compute $\bar{d}$ for the receiving parent, and add $\bar{d}n$ to the total computational effort. For regular mutation, compute $\bar{d}$ for the parent, and add $2\bar{d}n$ to the total computational effort. For aligned crossover, compute $\bar{d}$ for the receiving parent, and add $\bar{d}n \log n]$ to the total computational effort. For aligned insert mutation add $\bar{d}n)1 + \log n)$ to the total computational effort.